The Theory of Computing

Formal Alphabets and Formal Languages

Definition: An alphabet is a set of characters.

Examples:

$$\Sigma_{UC} = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z\}$$

$$\Sigma_{LC} = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, , u, v, w, x, y, z\}$$

$$\Gamma = \{\alpha, \beta, \gamma, \delta, \epsilon, \zeta, \eta, \theta, \iota, \kappa, \lambda, \mu, \nu, \xi, o, \pi, \rho, \sigma, \tau, \upsilon, \phi, \chi, \psi, \omega\}$$

$$\Sigma_{\text{DNA}} = \{G, A, T, C\}$$

$$\Sigma_{10} = \{0,1,2,3,4,5,6,7,8,9\}$$

We use capital Greek letters for names of alphabets.

$\Sigma = \{0,1\}$ will be our standard alphabet to study languages.

Definition: A string is a sequence of characters from an alphabet.

Examples:

01101110 is a string from $\Sigma = \{0,1\}$

$abacaa$ is a string from $\Gamma = \{a, b, c\}$

$GATTACA$ is a string from $\Delta = \{G, A, T, C\}$

Definition: The empty string, denoted ϵ or λ, is the string of length 0.

Definition: The set Σ^*, called the Kleene* closure of the set Σ, is the set of all strings over an alphabet Σ.



*named after Stephen Cole Kleene (1909-1994)

Example:

Using the alphabet $\Sigma = \{0,1\}$ we get:

$$\Sigma^* = \{\epsilon, 0,1,00,01,10,11,100,101,110,111, \dots\}$$

Definition: A formal language is a set of strings over an alphabet.

Examples: Given $\Sigma = \{0,1\}$:

$$L_0 = \{\epsilon, 0,1,00,01,10,11\}$$
$$L_1 = \{w \in \Sigma^* | \, w \text{ has an even length}\}$$
$$L_2 = \{0^n 1^n \mid n \geq 0\}$$
$$L_3 = \{w \in \Sigma^* \mid w \text{ has more 0s than 1s}\}$$
$$L_4 = \{w \in \Sigma^* | \, w \text{ is a palindrome}\}$$
$$L_5 = \left\{0^{n^2} \mid n \geq 0\right\}$$
$$L_6 = \{w \in \Sigma^* | \, w \text{ is a prime number}\}$$

Definition: The empty language, denoted Ø or {}, is a language with 0 strings.

A language membership problem is a problem of detecting if a string w is in a language L.

Classification problems like text, speech or image classification can be reduced to language membership problems.

The Theory of Computing

Deterministic Finite Automata

Computation is an information process that takes input information and processes it to produce output information.

Language Membership (for a given L)

Input: $w \in \Sigma^*$ a string over an alphabet

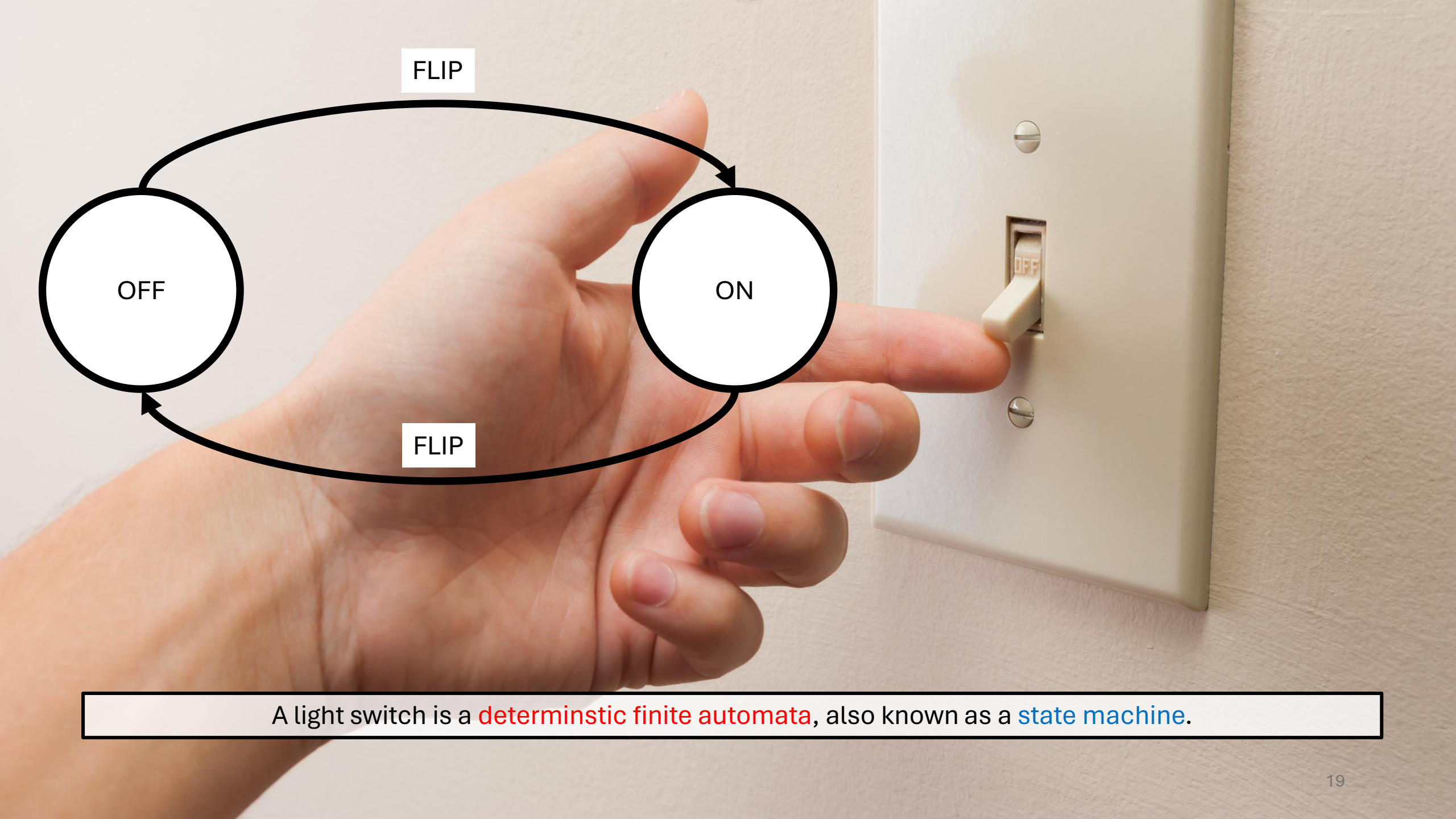
Output: "yes" if $w \in L$

"no" if $w \notin L$

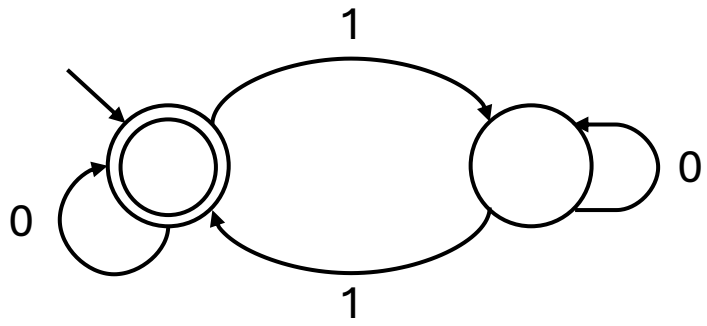
A problem describes the relationship between the input and output, and a computation describes the process of transforming the input into the output.

A model of computation is a description of the permissible information processing operations and their costs.

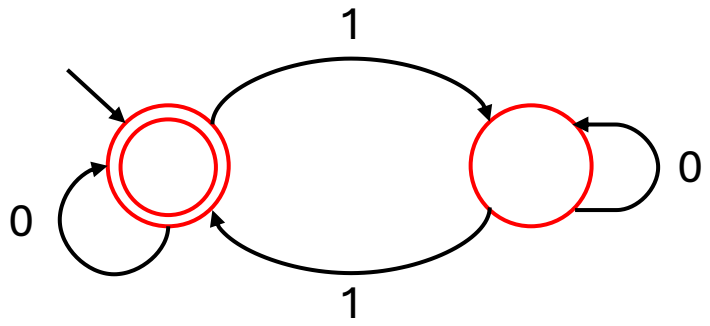
A simple model of computation is the deterministic finite automaton.



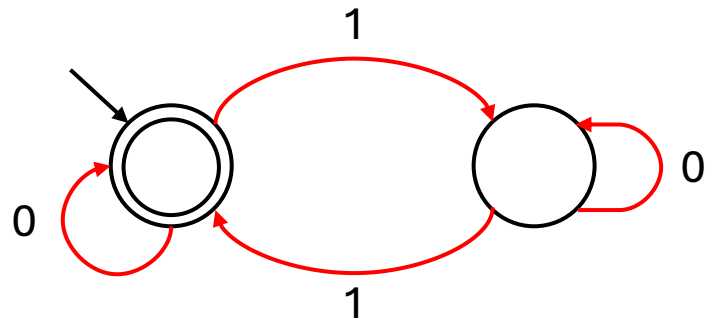
A light switch is a determinstic finite automata, also known as a state machine.



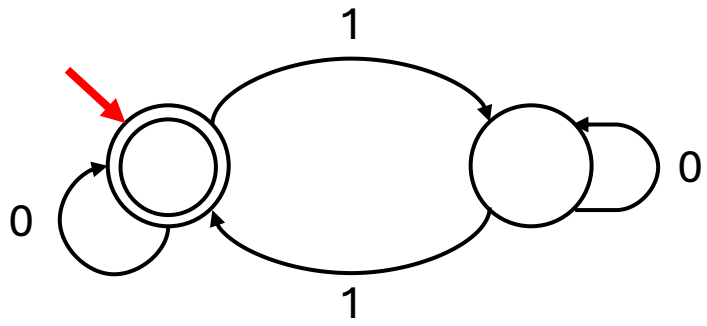
This is a deterministic finite automaton, M_1, that processes input strings over the alphabet $\Sigma = \{0,1\}$.



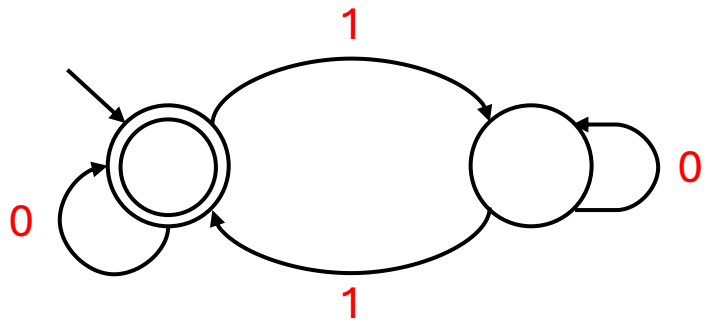
These are states.



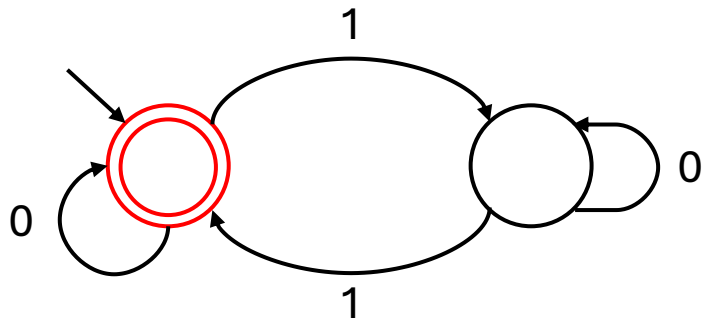
These are transitions between states.



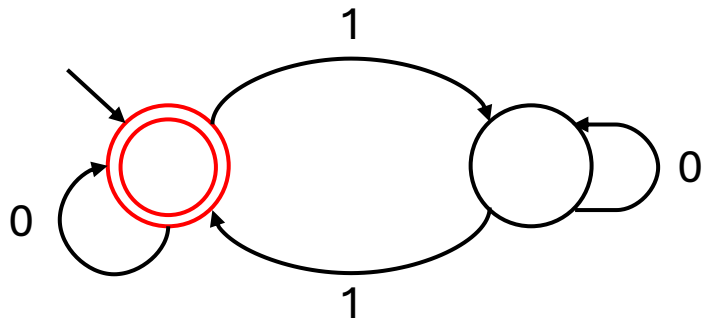
The red arrow points at the initial state where input processing begins.



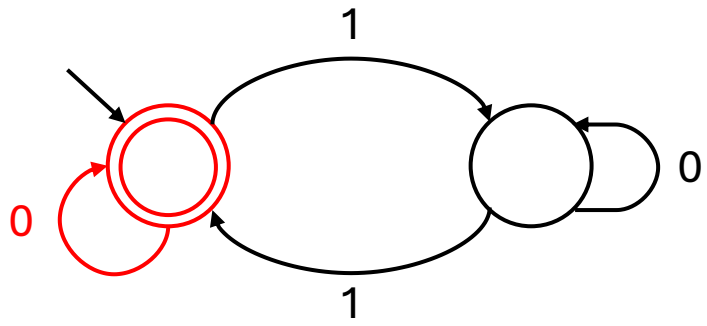
The automaton processes the input one character at a time by following the state transitions labelled with the input character.



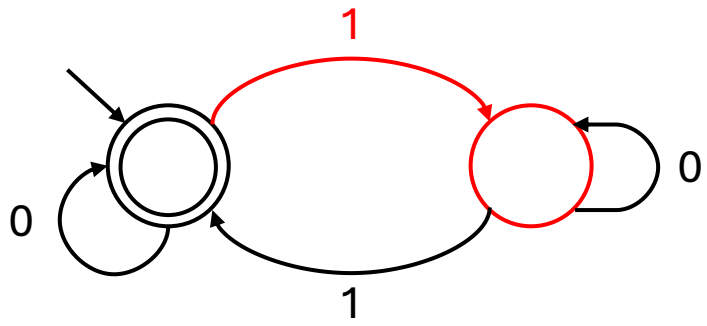
The double circle indicates that this is an accepting state. If processing ends in this state, the input is accepted. Otherwise, it is rejected.



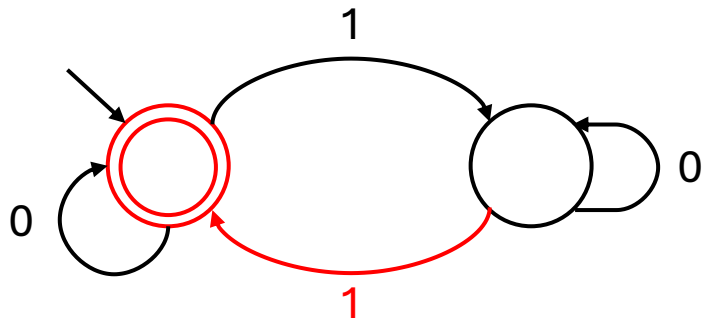
Let’s process the input 0110.



Let’s process the input 0110.

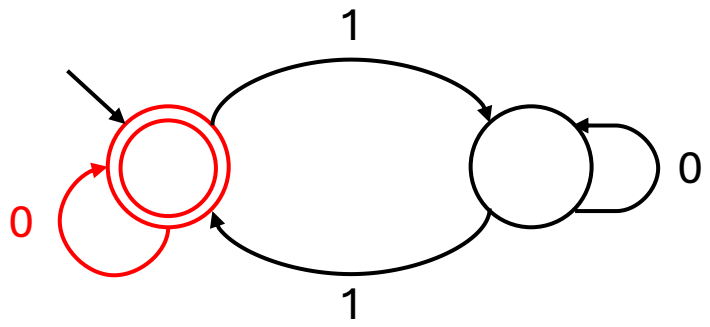


Let’s process the input 0110.

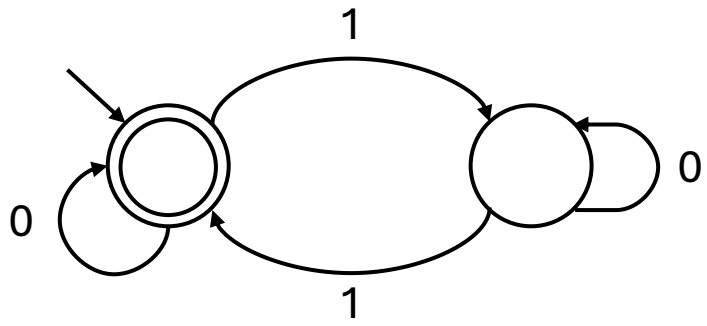


Let’s process the input 0110.

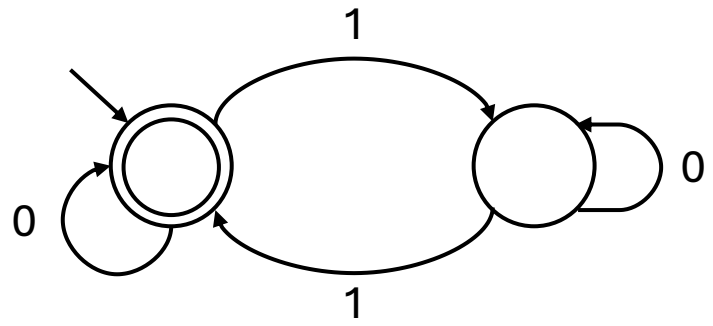
Accepted!



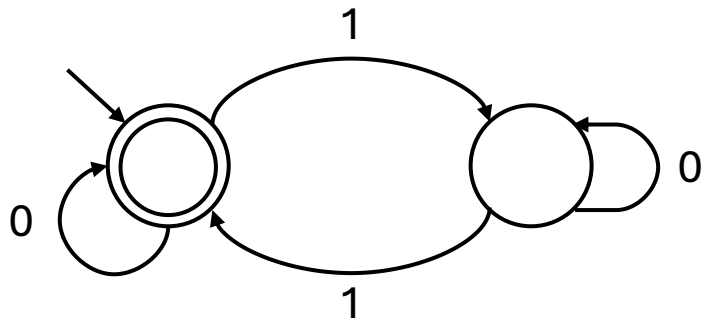
Let’s process the input 0110.



What is the language accepted by this machine?



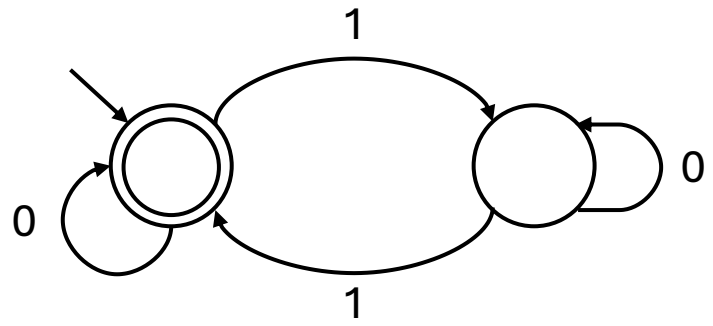
$L = \{w \in \{0,1\}^* \mid w$ has an even number of 1s$\}$



When a deterministic finite automata accepts the strings in a language, we say that it recognizes the language.

The Theory of Computing

Deterministic Finite Automata: Formal Definition



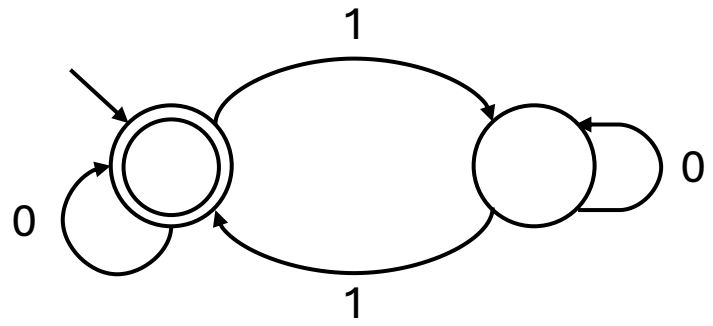
$L = \{w \in \{0,1\}^* \mid w$ has an even number of 1s$\}$

Definition: A deterministic finite automaton (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

1. Q is a finite set of *states*,
2. Σ is a finite *alphabet*,
3. $\delta: Q \times \Sigma \to Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states* (or *final states*).

A deterministic finite automaton computes as follows:

1. Initialization: The DFA begins processing in state q_o.
2. Computation: If the DFA is in state $q \in Q$ and is processing character $\sigma \in \Sigma$, then the next state is given by $\delta(q, \sigma)$.
3. Termination: Once all characters are processed if the DFA is in a state $q \in F$ the automaton accepts, otherwise it rejects.



We can describe the machine formally as $M_1 = \{Q, \Sigma, \delta, q_0, F\}$ where:

$Q = \{q_0, q_1\}$

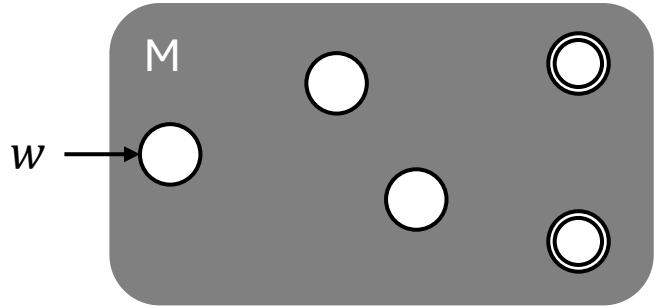
$\Sigma = \{0,1\}$

q_0 is the start state.

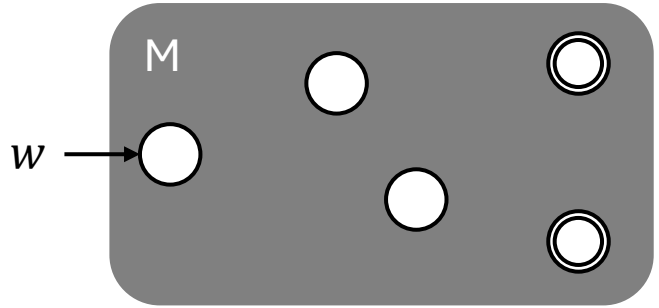
$F = \{q_0\}$

The transition function δ can be described by a table.

δ	0	1
q_0	q_0	q_1
q_1	q_1	q_0

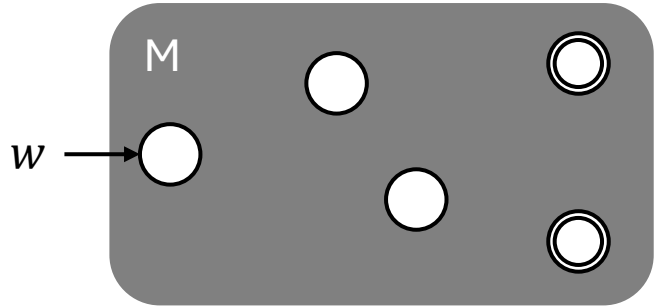


Let $M = (Q, \Sigma, \delta, q_o, F)$ be a DFA and let $w = w_1 w_2 \cdots w_n \in \Sigma^*$.



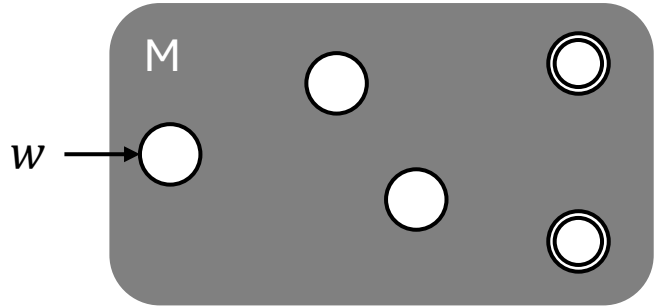
Definition: M accepts w if there exists a sequence of states, called a computation, $r_0, r_1, \ldots, r_n \in Q$ such that:

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, for $0, \ldots, n-1$, and
3. $r_n \in F$.



Definition: We say that M recognizes L, denoted $L(M) = L$, if

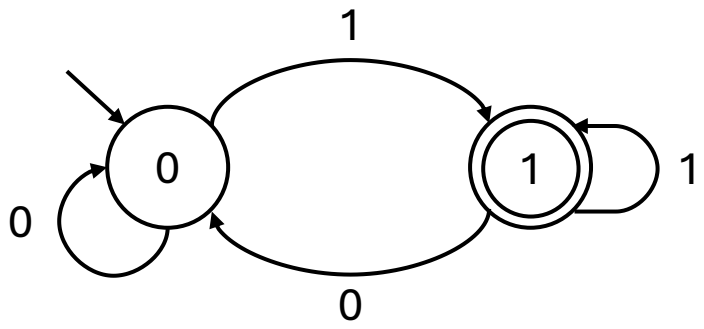
$$L = \{w \mid M \text{ accepts } w\}$$



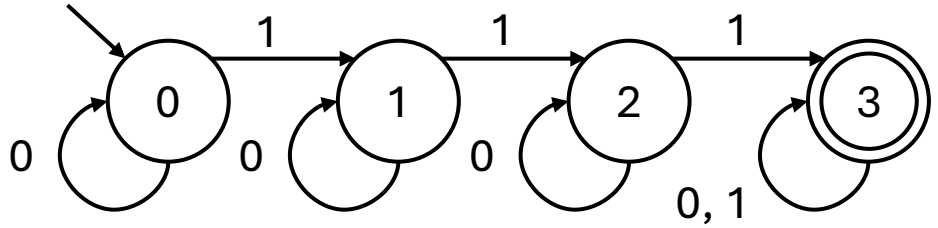
Definition: A language, L, is called a regular language, if $L = L(M)$, for some deterministic finite automaton M.

The Theory of Computing

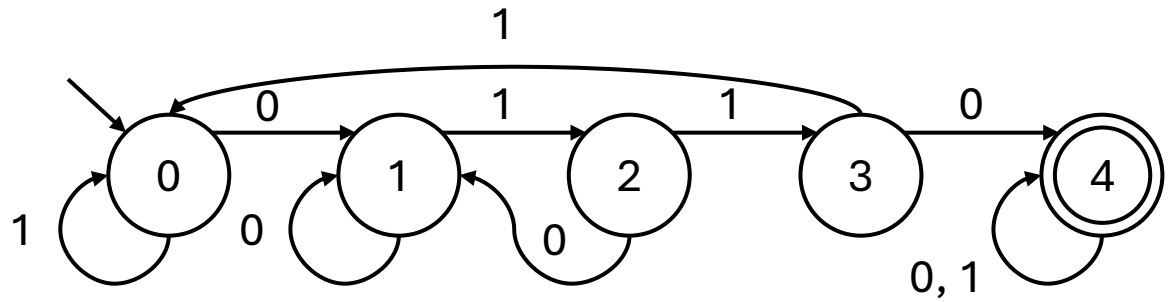
Deterministic Finite Automata: Examples



$L = \{w \in \{0,1\}^* \mid w \text{ ends with a } 1\}$



$L = \{w \in \{0,1\}^* \mid w \text{ has at least three 1s}\}$



$L = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 0110\}$

Exercises

$L_0 = \{w \in \{0,1\}^* \mid w \text{ ends with a } 0\}$

$L_1 = \{w \in \{0,1\}^* \mid w \text{ starts and ends with a } 0\}$

$L_2 = \{w \in \{0,1\}^* \mid w \text{ has at most three 1s}\}$

$L_3 = \{w \in \{0,1\}^* \mid w \text{ has exactly three 1s}\}$

$L_4 = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 10101\}$

$L_5 = \{w \in \{0,1\}^* \mid w \text{ does not contain the substring } 0101\}$

The Theory of Computing

Deterministic Finite Automata: Examples

Exercises

$L_0 = \{w \in \{a, b, c\}^* \mid w \text{ starts with } ab \text{ and ends with } ab\}$
$L_1 = \{w \in \{a, b, c\}^* \mid w \text{ contains at least one } a, b, \text{ and } c\}$
$L_2 = \{w \in \{0,1\}^* \mid |w| \text{ is divisible by } 3\}$
$L_3 = \{w \in \{0,1\}^* \mid w \text{ starts with } 0 \text{ or } |w| \text{ is divisible by } 5\}$
$L_4 = \{w \in \{0,1\}^* \mid w \text{ has four } 0\text{s}\}$
$L_5 = \{w \in \{0,1\}^* \mid w \text{ has two } 0\text{s and two } 1\text{s}\}$
$L_6 = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 1110\}$
$L_7 = \{w \in \{0,1\}^* \mid w \text{ does not contain the substring } 0011\}$
$L_8 = \{w \in \{0,1\}^* \mid w \text{ does not contain the substring } 10011\}$
$L_9 = \{w \in \{a, b, c\}^* \mid w \text{ contains the substring } abc \text{ at least twice}\}$
$L_{10} = \{w \in \{a, b, c\}^* \mid w \text{ contains at least one } a \text{ and an even number of } c\text{s}\}$